

SQL MASTERY

1000+ Practice Questions

RetailDB — A Comprehensive SQL Server Practice Database

EASY 95 Qs

MEDIUM 185 Qs

HARD 250 Qs

EXPERT 50 Qs

About This Book

This question bank is designed for the RetailDB SQL Server practice database — a realistic multi-table retail schema covering customers, orders, products, employees, stores, inventory and more. Work through all four levels to go from solid fundamentals to expert-level data engineering and analytics.

How to Use This Book

1. Set up RetailDB using the provided Schema + Data SQL script.
2. Attempt each question in your SQL Server environment before looking at hints.
3. Time yourself — aim for <2 min (Easy), <5 min (Medium), <15 min (Hard), <30 min (Expert).
4. Focus on execution plans for Hard and Expert questions to understand performance.
5. For Expert questions, also write a brief explanation of your approach.

Table of Contents

Chapter 1	Database Overview & RetailDB Schema	3
Chapter 2	EASY Level Questions (Q1 – Q95)	6
2.1	SELECT & Basic Queries (Q1–Q30)	6
2.2	Filtering Data (Q31–Q65)	8
2.3	Aggregate Functions (Q66–Q85)	11
2.4	Sorting & TOP (Q86–Q95)	13
Chapter 3	MEDIUM Level Questions (Q96 – Q280)	14
3.1	JOIN Statements (Q96–Q120)	14
3.2	Aggregation & Grouping (Q121–Q140)	18
3.3	Subqueries (Q141–Q155)	21
3.4	String Functions (Q156–Q170)	23
3.5	Date & Time Functions (Q171–Q185)	25
3.6	NULL Handling (Q186–Q195)	27
3.7	Combined Challenges (Q196–Q280)	28
Chapter 4	HARD Level Questions (Q281 – Q530)	35
4.1	Window Functions (Q281–Q300)	35
4.2	CTEs & Recursive Queries (Q301–Q315)	39
4.3	Advanced JOINS & Set Operations (Q316–Q330)	42
4.4	Business Analytics Queries (Q331–Q345)	45
4.5	Complex Multi-Table Scenarios (Q346–Q530)	48
Chapter 5	EXPERT Level Questions (Q531–Q580+)	65
5.1	Complex Analytics & Modelling (Q531–Q545)	65
5.2	Performance & Optimisation (Q546–Q550)	69
5.3	Advanced DML & Transactions (Q551–Q555)	70
5.4	Real-World Scenarios (Q556–Q580+)	71

Chapter 1 — RetailDB Database Overview

Table	Key Columns	Purpose
Regions	RegionID, RegionName, Country	Geographic regions
Stores	StoreID, StoreName, RegionID, StoreType	Physical & online stores
Departments	DepartmentID, DeptName, Budget	Company departments
Employees	EmployeeID, FullName, DeptID, StoreID, ManagerID, Salary	Staff (self-referencing hierarchy)
Categories	CategoryID, CategoryName, ParentCategoryID	Product hierarchy (recursive)
Suppliers	SupplierID, SupplierName, Country, Rating	Product suppliers
Products	ProductID, ProductName, CategoryID, SupplierID, UnitPrice, Cost, Price, Stock	Product catalog
Customers	CustomerID, FullName, Email, City, JoinDate, LoyaltyPoints	Customer master
Orders	OrderID, CustomerID, StoreID, EmployeeID, OrderDate, Status, PaymentMethod	Sales orders
OrderDetails	OrderDetailID, OrderID, ProductID, Quantity, UnitPrice, Discount	Order items per order
Inventory	InventoryID, ProductID, StoreID, Quantity	Stock per store
Returns	ReturnID, OrderDetailID, Reason, RefundAmount, Status	Product returns
Reviews	ReviewID, ProductID, CustomerID, Rating, ReviewText, ReviewDate	Customer reviews
Promotions	PromotionID, PromoName, DiscountPct, StartDate, EndDate	Discount campaigns
EmployeeShifts	ShiftID, EmployeeID, ShiftDate, StartTime, EndTime, StoreID	Work schedules

Data Volumes (after running the data generation script)

Customers: 2,000+ rows | Products: 500+ rows | Orders: 10,000+ rows | OrderDetails: 30,000+ rows | Employees: 200+ rows

Use the companion Python data-generation script to populate the database with realistic data.

Chapter 2 — EASY Level Questions

These questions test fundamental SQL: SELECT, basic filtering, simple aggregations and sorting. You should be able to answer each in under 2 minutes.

2.1 SELECT & Basic Queries

- Q1.** Select all columns from the Customers table.
- Q2.** Select only the FirstName and LastName from Customers.
- Q3.** Select all products from the Products table.
- Q4.** Display the ProductName and UnitPrice of all products.
- Q5.** List all employees with their JobTitle.
- Q6.** Show all orders from the Orders table.
- Q7.** Display the OrderID, OrderDate and Status of every order.
- Q8.** Select all columns from the Stores table.
- Q9.** Show all suppliers with their country.
- Q10.** List all departments and their budgets.
- Q11.** Select the Email and Phone of all customers.
- Q12.** Show the HireDate and Salary of all employees.
- Q13.** Display all product categories.
- Q14.** List all regions and countries.
- Q15.** Show the first 10 rows from the Orders table.
- Q16.** Retrieve the top 5 most expensive products by UnitPrice.
- Q17.** Select the top 20 customers ordered by JoinDate.
- Q18.** Display the top 10 orders by OrderDate descending.
- Q19.** Show the first 15 employees ordered alphabetically by LastName.
- Q20.** Select the distinct countries from the Customers table.
- Q21.** List all distinct job titles from the Employees table.
- Q22.** Show all distinct payment methods used in Orders.
- Q23.** Display distinct order statuses from the Orders table.
- Q24.** List distinct customer tiers from the Customers table.
- Q25.** Show all distinct store types from the Stores table.
- Q26.** Count the total number of customers.

- Q27. Count how many products are in the Products table.
- Q28. Count all orders in the Orders table.
- Q29. Count distinct countries in the Suppliers table.
- Q30. How many employees are stored in the database?

2.2 Filtering Data

- Q31. Find all customers from 'USA'.
- Q32. Find all products with UnitPrice greater than 50.
- Q33. List employees with Salary less than 40000.
- Q34. Show all orders with Status = 'Delivered'.
- Q35. Find all customers whose CustomerTier is 'Gold'.
- Q36. List products that are discontinued (Discontinued = 1).
- Q37. Show employees hired after '2018-01-01'.
- Q38. Find all orders placed in the year 2023.
- Q39. List customers with LoyaltyPoints greater than 500.
- Q40. Show products with StockQuantity below 20.
- Q41. Find all stores of type 'Online'.
- Q42. Show orders with PaymentMethod = 'Card'.
- Q43. List employees in DepartmentID = 1 (Sales).
- Q44. Find customers born before 1990.
- Q45. List all products where CostPrice is NULL.
- Q46. Show employees whose Salary is between 30000 and 60000.
- Q47. Find orders placed between '2022-01-01' and '2022-12-31'.
- Q48. List products with UnitPrice between 10 and 100.
- Q49. Show customers with LoyaltyPoints between 100 and 1000.
- Q50. Find employees with Salary NOT between 50000 and 80000.
- Q51. Find customers whose LastName starts with 'S'.
- Q52. List products whose ProductName contains 'Pro'.
- Q53. Show employees whose Email ends with '@company.com'.
- Q54. Find customers whose city is either 'New York' or 'London'.
- Q55. List orders where Status IN ('Pending', 'Processing').

Q56. Show products supplied by SupplierID IN (1, 2, 3).

Q57. Find employees in departments 2, 3, or 5.

Q58. List customers NOT from 'USA'.

Q59. Show orders with Status NOT IN ('Cancelled', 'Returned').

Q60. Find products NOT in CategoryID 1.

Q61. List employees where ManagerID IS NULL (top-level managers).

Q62. Find customers with no phone number (Phone IS NULL).

Q63. Show products with a non-null Description.

Q64. List orders where ShippedDate IS NULL.

Q65. Find suppliers with a NULL Rating.

2.3 Aggregate Functions

Q66. Find the maximum UnitPrice among all products.

Q67. Find the minimum Salary among all employees.

Q68. Calculate the average UnitPrice of all products.

Q69. Calculate the total (SUM) of all salaries.

Q70. Count how many products are in CategoryID = 1.

Q71. Find the highest LoyaltyPoints among all customers.

Q72. Find the average Salary grouped by DepartmentID.

Q73. Count the number of orders per Status.

Q74. Sum the StockQuantity by CategoryID.

Q75. Find the minimum and maximum UnitPrice per category.

Q76. Count how many customers joined each year.

Q77. Find the average rating per product using the Reviews table.

Q78. Count orders per StoreID.

Q79. Sum the total Salary per JobTitle.

Q80. Find the maximum salary per department.

Q81. Count how many employees each manager supervises.

Q82. Show CategoryID groups with more than 5 products (HAVING).

Q83. Show DepartmentIDs where the average salary > 50000.

Q84. List StoreIDs with more than 100 orders.

Q85. Find job titles where the maximum salary exceeds 90000.

2.4 Sorting & TOP

Q86. List all products ordered by UnitPrice ascending.

Q87. List all employees ordered by Salary descending.

Q88. Show orders sorted by OrderDate descending.

Q89. Display customers sorted by LastName, then FirstName.

Q90. List products sorted by CategoryID ascending, then ProductName ascending.

Q91. Show employees sorted by HireDate ascending.

Q92. Display suppliers sorted by Rating descending.

Q93. List orders sorted by Status ascending, then OrderDate descending.

Q94. Show customers sorted by LoyaltyPoints descending.

Q95. List reviews sorted by Rating descending, ReviewDate descending.

Chapter 3 — MEDIUM Level Questions

Medium questions require JOINS, subqueries, grouped aggregations and string/date functions. Target under 5 minutes each.

3.1 JOIN Statements

- Q96.** List each order with the customer's full name.
- Q97.** Show each order with the store name where it was placed.
- Q98.** Display order details including the product name and quantity.
- Q99.** List employees with their department name.
- Q100.** Show products with their category name.
- Q101.** Display products with their supplier name and country.
- Q102.** List all orders with the employee's full name who handled them.
- Q103.** Show order details with product name, quantity and total line price (Quantity * UnitPrice).
- Q104.** List customers with their total number of orders.
- Q105.** Display each store with its region name and country.
- Q106.** Show employees with their manager's full name.
- Q107.** List all products with their category and supplier name.
- Q108.** Display orders with customer name, store name and order total.
- Q109.** Show all reviews with the customer name and product name.
- Q110.** List inventory records with store name and product name.
- Q111.** Show returns with the product name and reason.
- Q112.** Display promotions that are currently active (today between StartDate and EndDate).
- Q113.** List order details where the line discount > 10.

Q11 Show employees who work at the 'Online Store'.
4.

Q11 List customers who have never placed an order (LEFT JOIN approach).
5.

Q11 Show products that have never been ordered.
6.

Q11 Find employees who have never managed anyone.
7.

Q11 List stores that have no inventory records.
8.

Q11 Show customers who have written at least one review.
9.

Q12 Display products reviewed by more than 10 customers.
0.

3.2 Aggregation & Grouping

Q12 Find the total revenue per store (SUM of Quantity * UnitPrice in OrderDetails).
1.

Q12 Calculate the total revenue per customer.
2.

Q12 Show the top 10 best-selling products by total units sold.
3.

Q12 Find the average order value per customer.
4.

Q12 Calculate total revenue by year.
5.

Q12 Show monthly sales totals for the year 2023.
6.

Q12 Find the total revenue by product category.
7.

Q12 Calculate the profit margin per product $((\text{UnitPrice} - \text{CostPrice}) / \text{UnitPrice} * 100)$.
8.

Q12 Show total orders per month per store.
9.

Q13 Find employees with the highest total sales revenue.
0.

Q13 Calculate the average discount given per order.
1.

Q13 Show the count and total revenue of orders per payment method.
2.

Q13 Find the most popular product category by units sold.
3.

Q13 Show total loyalty points earned per customer tier.
4.

Q13 Calculate the average salary per store.
5.

Q13 Find departments where average salary exceeds the company-wide average.
6.

Q13 Show stores with revenue above the average store revenue.
7.

Q13 List customers whose total spending exceeds \$5000.
8.

Q13 Find products where total revenue > \$10,000.
9.

Q14 Calculate year-over-year revenue growth.
0.

3.3 Subqueries

Q14 Find products priced above the average product price.
1.

Q14 List customers who have spent more than the average customer spending.
2.

Q14 Show employees earning more than the average salary in their department.
3.

Q14 Find the most expensive product in each category using a subquery.
4.

Q14 List orders with a total value above the average order value.
5.

Q14 Show products that have been returned at least once.
6.

Q14 Find customers who placed orders in every year from 2021 to 2023.
7.

Q14 List employees whose salary is in the top 10% of all salaries.
8.

Q14 Show stores whose revenue is above the median store revenue.
9.

Q15 Find products that have a higher rating than the category average.
0.

Q15 List customers who bought products from more than 3 different categories.
1.

Q15 Show suppliers who supply the most expensive product in their category.
2.

Q15 Find the second highest salary in the Employees table.
3.

Q15 List employees whose salary is higher than their manager's salary.
4.

Q15 Show orders that contain the product 'ProductCode = PRD001'.
5.

3.4 String Functions

Q15 Concatenate FirstName and LastName as FullName for all employees.
6.

Q15 Convert all customer email addresses to uppercase.
7.

Q15 Extract the domain part from customer emails (after '@').
8.

Q15 Show the first 3 characters of each product code.
9.

Q16 Find customers whose name length exceeds 10 characters.
0.

Q16 Replace 'Street' with 'St.' in store city names.
1.

Q16 Show employees with a reversed first name.
2.

Q16 Trim whitespace from supplier contact names.
3.

Q16 Find products whose name starts with a vowel.
4.

Q16 Show the length of each product's description.
5.

Q16 Format product prices as strings with 2 decimal places.
6.

Q16 Show customers whose email contains 'gmail'.
7.

Q16 Split the ProductCode to get only the numeric portion.
8.

Q16 Concatenate StoreName and City as 'StoreName – City'.
9.

Q17 Find all employees whose last name contains exactly 5 characters.
0.

3.5 Date & Time Functions

Q17 Calculate each employee's tenure in years from HireDate to today.
1.

Q17 Show orders placed in the last 30 days.
2.

Q17 Extract the month and year from OrderDate.
3.

Q17 Find employees hired in the first quarter (Jan-Mar) of any year.
4.

Q17 Show customers who joined in the last 6 months.
5.

Q17 Calculate the days between OrderDate and ShippedDate for each order.
6.

Q17 Find orders that took more than 7 days to ship.
7.

Q17 Show the day of week for each order date.
8.

Q17 Find customers whose birthday is this month.
9.

Q18 Calculate the age of each customer based on BirthDate.
0.

Q18 Show orders placed on weekends.
1.

Q18 Find products added (via first order) more than 2 years ago.
2.

Q18 Show the most recent order date per customer.
3.

Q18 Calculate how long each promotion ran in days.
4.

Q18 Find all orders where ShippedDate was after the RequiredDate.
5.

3.6 NULL Handling

- Q18 6.** Show products replacing NULL CostPrice with 0.
- Q18 7.** Display employee names using COALESCE for ManagerID (show 'Top Manager' if NULL).
- Q18 8.** Show orders where ShippedDate is NULL and Status is 'Processing'.
- Q18 9.** Replace NULL descriptions with 'No description available'.
- Q19 0.** Find the count of products with and without a description.
- Q19 1.** Use ISNULL to replace NULL phone numbers with 'N/A' in Customers.
- Q19 2.** Show products where Weight is unknown (NULL).
- Q19 3.** Handle NULL discounts in OrderDetails treating them as 0.
- Q19 4.** Find orders where RequiredDate is NULL.
- Q19 5.** Show the percentage of employees with missing phone numbers.

3.7 Combined Challenges

- Q19 6.** Show total revenue by store per quarter for 2023.
- Q19 7.** Find customers who placed more than 5 orders but have a CustomerTier of 'Bronze'.
- Q19 8.** List employees who earn more than the average salary of their job title.
- Q19 9.** Show the 3 most recent orders for each customer.
- Q20 0.** Find stores where the total number of returned items exceeds 50.
- Q20 1.** Calculate the percentage of orders that were cancelled per store.
- Q20 2.** List products where the average review rating is below 3 and total units sold > 100.

Q20 Show the employee with the highest sales in each department.
3.

Q20 Find customers who have ordered every product in CategoryID = 1.
4.

Q20 Calculate the average time (in days) from OrderDate to ShippedDate by store.
5.

Q20 Show products whose stock has fallen below the reorder level.
6.

Q20 List promotions that were never used in any order.
7.

Q20 Find the top 5 cities by total customer spending.
8.

Q20 Show customers who have placed orders in at least 3 different stores.
9.

Q21 Calculate the revenue impact if all Bronze-tier customers upgraded to Silver (assuming 10% more spending).
0.

Q21 Find employees who have worked shifts in more than one store.
1.

Q21 Show the most common payment method per store.
2.

Q21 List products that have been reviewed but never ordered (data quality check).
3.

Q21 Find customers whose average order value exceeds \$500.
4.

Q21 Show the total discount given per promotion.
5.

Q21 Calculate the revenue per employee for the Sales department.
6.

Q21 Find suppliers whose average product price is above \$75.
7.

Q21 List stores with an average order value above the overall average.
8.

Q21 Show the month with the highest number of new customer registrations.
9.

Q22 Calculate what percentage of total revenue comes from Gold and Platinum customers.
0.

Q22 Find products that generated the most returns by value.
1.

Q22 2.	Show the city with the most customers.
Q22 3.	List employees who have been with the company for more than 5 years and earn less than \$50,000.
Q22 4.	Find the product with the highest profit margin.
Q22 5.	Show total units sold per supplier.
Q22 6.	Calculate the average number of products per order.
Q22 7.	Find customers who have never given a review despite having placed 3+ orders.
Q22 8.	Show which day of the week generates the most revenue on average.
Q22 9.	List all products that appear in returns more than twice.
Q23 0.	Calculate cumulative revenue per month for the current year.
Q23 1.	Find stores whose revenue has grown in each of the last 3 years.
Q23 2.	Show the top 10 employees by number of orders processed.
Q23 3.	Find product categories where average cost-to-price margin is below 20%.
Q23 4.	List customers who placed their first order within 7 days of joining.
Q23 5.	Find orders containing more than 5 unique products.
Q23 6.	Show suppliers that have not supplied a new product in the last 2 years.
Q23 7.	Calculate each store's share of total company revenue.
Q23 8.	Find employees who have processed orders for cancelled status more than 10 times.
Q23 9.	Show the top 3 most returned products per category.
Q24 0.	Calculate the average loyalty points earned per order.

Q24 1.	Find customers whose spending increased year over year.
Q24 2.	Show monthly revenue with a comparison label: 'Above Average' or 'Below Average'.
Q24 3.	List products where the number of 1-star reviews exceeds 5-star reviews.
Q24 4.	Find the store that has the highest employee-to-revenue ratio.
Q24 5.	Calculate total revenue for each combination of CustomerTier and PaymentMethod.
Q24 6.	Find the product that has the longest average time between repeat purchases.
Q24 7.	List customers who have shopped at every store (excluding the Online Store).
Q24 8.	Show year-over-year change in total orders and revenue side-by-side.
Q24 9.	Find employees whose salary is above average for their store AND their department.
Q25 0.	Calculate the return rate by payment method.
Q25 1.	Show the distribution of order values by bucket: <\$50, \$50-\$200, \$200-\$500, \$500+.
Q25 2.	Find the best and worst performing product in each category by revenue.
Q25 3.	List customers who placed orders in consecutive months for at least 6 months.
Q25 4.	Calculate total and average order value by store type.
Q25 5.	Show the top 5 employees with the highest average basket value per transaction.
Q25 6.	Find products frequently purchased together with a specific product (e.g., ProductID = 10).
Q25 7.	Calculate the number of active customers (at least 1 order) per quarter.
Q25 8.	Show orders that include both ProductID = 5 and ProductID = 10.
Q25 9.	Find the average number of days between a customer's first and second order.

Q26 0.	List regions ranked by total revenue.
Q26 1.	Show each employee's total managed revenue including their subordinates' revenue.
Q26 2.	Find the product category with the fastest growth in revenue between 2022 and 2023.
Q26 3.	Calculate customer tier upgrade potential: Bronze customers who spent > \$2000 last year.
Q26 4.	Show the 10 customers who have been waiting the longest for an order to ship.
Q26 5.	Find stores where more than 30% of orders are from repeat customers.
Q26 6.	Calculate the average review rating for products by price bracket.
Q26 7.	Show the employee hierarchy depth using a recursive CTE (count levels).
Q26 8.	Find any duplicate customer emails (potential data issues).
Q26 9.	Show total orders and revenue for each combination of StoreType and PaymentMethod.
Q27 0.	Find products that have been ordered every single month in the past year.
Q27 1.	Calculate the 'win back' opportunity: customers who were active in 2022 but placed no orders in 2023.
Q27 2.	Show how many days on average it took to fulfil orders by store, by year.
Q27 3.	Find the top 3 revenue-generating employees in each store.
Q27 4.	Calculate percentage of revenue from new vs returning customers per quarter.
Q27 5.	Show products with a review count greater than the category average.
Q27 6.	Find customers who only ever buy from one specific store.

Chapter 4 — HARD Level Questions

Hard questions require mastery of window functions, CTEs, recursive queries and multi-step business analytics. Target under 15 minutes each. Always check your execution plan.

4.1 Window Functions

Q27 Rank customers by total spending using RANK().
7.

Q27 Use DENSE_RANK() to rank products within each category by UnitPrice.
8.

Q27 Calculate a running total of daily revenue using SUM() OVER (ORDER BY OrderDate).
9.

Q28 Find the top 3 selling products in each category using ROW_NUMBER().
0.

Q28 Calculate each employee's salary percentile using PERCENT_RANK().
1.

Q28 Use LAG() to compare each month's revenue to the previous month.
2.

Q28 Use LEAD() to show the next order date for each customer.
3.

Q28 Calculate a 3-month moving average of sales revenue.
4.

Q28 Find the first and last order date per customer using FIRST_VALUE() and LAST_VALUE().
5.

Q28 Assign quartiles to employees by salary using NTILE(4).
6.

Q28 Calculate cumulative sales per store over time using SUM() OVER (PARTITION BY StoreID ORDER BY OrderDate).
7.

Q28 Show each order's rank within its store by order value.
8.

Q28 Calculate the difference between each product's price and the category average using AVG() OVER (PARTITION BY CategoryID).
9.

Q29 Use ROW_NUMBER() to de-duplicate customers with duplicate emails (keep the earliest).
0.

Q29 Show running count of orders per customer over time.
1.

Q29 Find months where revenue dropped compared to the previous month using LAG().
2.

Q29 Calculate each employee's salary as a percentage of the department total.
3.

Q29 Use NTILE(10) to bucket customers into deciles by total spending.
4.

Q29 Compute a 7-day rolling average of daily order counts.
5.

Q29 Use CUME_DIST() to find the cumulative distribution of product prices.
6.

4.2 CTEs & Recursive Queries

Q29 Use a CTE to find the top 5 customers by lifetime value.
7.

Q29 Write a recursive CTE to display the full employee reporting hierarchy.
8.

Q29 Use a CTE to calculate monthly revenue then compare to the annual average.
9.

Q30 Create a CTE that finds products never ordered and another that finds slow movers, then UNION them.
0.

Q30 Use a CTE to identify customers at risk of churn (no order in 180 days).
1.

Q30 Write a CTE chain to compute: orders -> order totals -> customer totals -> top customers.
2.

Q30 Use a recursive CTE to traverse the product category hierarchy.
3.

Q30 Create a CTE to calculate each store's market share of total revenue.
4.

Q30 Use a CTE to find the nth highest salary without using TOP.
5.

Q30 Write a CTE to identify which promotions resulted in the highest order values.
6.

Q30 Use multiple CTEs to compare revenue: current year vs prior year.
7.

Q30 Create a CTE that computes a 'health score' for each supplier (rating, order volume, return rate).
8.

Q30 Use a CTE to find the longest streak of consecutive days with at least one order.
9.

Q31 Write a CTE to calculate average order value per customer cohort (by join year).
0.

Q31 Create a recursive CTE that generates a sequence of dates for a given range.
1.

4.3 Advanced JOINS & Set Operations

Q31 Use a CROSS JOIN to generate all possible product-store combinations, then LEFT JOIN inventory to see which are stocked.
2.

Q31 Self-join the Employees table to show each employee paired with their manager's details.
3.

Q31 Join Orders to OrderDetails to Customers to Products in a single query to show full order breakdown.
4.

Q31 Use a non-equij join to find customers who joined before the stores they ordered from opened.
5.

Q31 Find pairs of products frequently bought together (SELF JOIN on OrderDetails by OrderID).
6.

Q31 Use a CROSS APPLY to get each customer's most recent order.
7.

Q31 Use OUTER APPLY to get the top 2 products per category by revenue.
8.

Q31 Join the Returns table to trace the full return path: Return -> OrderDetail -> Order -> Customer -> Product.
9.

Q32 Use a derived table (subquery in FROM) to pre-aggregate order totals then join to Customers.
0.

Q32 Perform a multi-table join to show employee shift details with store and region information.
1.

Q32 Find customers who ordered in both 2022 and 2023 using JOIN on self-aggregated data.
2.

Q32 Use EXISTS to list products that have received 5-star reviews.
3.

Q32 Use NOT EXISTS to find customers who have never returned a product.
4.

Q32 Join promotions to orders to calculate how many orders qualified for each promotion.
5.

Q32 Find products that are both top-selling AND top-rated using JOINS between aggregated CTEs.
6.

4.4 Business Analytics

Q32 Calculate month-over-month revenue growth rate as a percentage.
7.

Q32 Find the customer retention rate: % of customers who ordered in year N and also in year N+1.
8.

Q32 Compute the average number of days between consecutive orders per customer.
9.

Q33 Calculate the product return rate (returns / units sold) for each product.
0.

Q33 Find the revenue contribution of the top 20% of customers (Pareto analysis).
1.

Q33 Compute the average basket size (items per order) per store.
2.

Q33 Identify seasonality: which month has the highest average revenue across all years.
3.

Q33 Calculate inventory turnover ratio per product (Units Sold / Avg Stock).
4.

Q33 Find cross-sell pairs: products most frequently ordered together.
5.

Q33 Compute customer lifetime value (total spend) segmented by acquisition year.
6.

Q33 Identify the 'golden customers': top 10% by spend AND top 10% by order frequency.
7.

Q33 Calculate employee productivity: revenue generated per employee per store.
8.

Q33 Find the churn rate by customer tier (customers who stopped ordering).
9.

Q34 Compute the average review rating trend over time per product.
0.

Q34 Identify stores where revenue is declining over the last 3 consecutive months.
1.

4.5 Complex Multi-Table Scenarios

Q34 Identify all customers who have had a gap of more than 365 days between any two consecutive orders.
2.

Q34 Write a query that produces a complete sales funnel: total customers → customers who ordered once → customers who ordered 2+ times → customers who ordered 5+ times → VIP (10+ orders).
3.

Q34 Find the optimal reorder schedule for all products: calculate the average daily sales rate and determine how many days of stock remain.
4.

Q34 Build a complete store comparison matrix: for each pair of stores, calculate how many customers they share.
5.

Q34 6.	Using window functions only (no subqueries), find each customer's first, second and third most purchased product category.
Q34 7.	Write a query that assigns each order to a revenue quartile AND returns the interquartile range of order values per store.
Q34 8.	Find employees whose sales performance ranks in the top 25% in their store but below average company-wide.
Q34 9.	Calculate a 'product stickiness score' for each product: average orders per customer who has ever purchased it.
Q35 0.	Identify 'category champion' customers: the single customer who spent the most in each product category.
Q35 1.	Write a query to detect potential fraud: orders where the same customer placed more than 3 orders in a single day totalling over \$1000.
Q35 2.	Build a full cohort analysis: for customers who joined each quarter, show their average total spend at the 30-day, 90-day, 180-day and 365-day marks.
Q35 3.	Write a single query (no application code) to generate a complete product affinity matrix showing which products are most often bought together.
Q35 4.	Find the 'golden path': the most common sequence of 3 product categories customers buy in their first 3 orders.
Q35 5.	Detect inventory discrepancies: products where the sum of order quantities exceeds the initial stock plus any positive inventory adjustments.
Q35 6.	Write a query to calculate the Gini coefficient of customer spending (measure of spending inequality).
Q35 7.	Using GROUPING SETS, produce a single report showing revenue totalled by: each store, each region, each store type and a grand total.
Q35 8.	Build a loyalty tier escalation report: show customers who are within \$X of reaching the next tier, ordered by proximity.
Q35 9.	Find the product that most frequently appears as the 'last thing a customer bought' before going inactive (no further orders).
Q36 0.	Write a query to compute the rolling 12-month NPS equivalent (% 4-5 star reviews minus % 1-2 star reviews) per product.
Q36 1.	Identify stores where the product mix (categories sold) changed significantly between 2022 and 2023 (at least 3 categories that were top-5 in 2022 are not top-5 in 2023).
Q36 2.	Calculate each supplier's 'reliability score': on-time delivery rate based on Orders where ShippedDate <= RequiredDate for their products.
Q36 3.	Find customer pairs who have purchased exactly the same set of products.
Q36 4.	Write a query to compute the average customer acquisition cost proxy: total promotion discounts given divided by new customers acquired per quarter.

Q36 5.	Implement a simple moving average crossover signal: flag months where the 3-month MA crosses above or below the 6-month MA of revenue.
Q36 6.	Find employees who are simultaneously in the top 10 by orders processed AND the bottom 20% by average order value — potential efficiency vs quality issue.
Q36 7.	Build a geographic revenue heat map query: show revenue per city sorted by latitude proxy (city name alphabetically as a stand-in).
Q36 8.	Calculate the product cannibalization ratio: for each new product launch, measure the % revenue decline of the most similar existing product in the same category.
Q36 9.	Find the minimum number of products a customer must buy to account for 80% of their personal total spend (personal Pareto).
Q37 0.	Write a query that detects 'review bombing': products that received 5 or more 1-star reviews within a 7-day window.
Q37 1.	Calculate daily revenue per store with weekend vs weekday breakdown and percentage difference.
Q37 2.	Find the top 3 most profitable promotions based on: (incremental revenue during promotion) vs (total discount cost).
Q37 3.	Build a customer journey map: for each customer, list their orders in sequence with the time gap (days) between each and the running total spent.
Q37 4.	Identify 'dark stores': stores with inventory but zero orders in the last 90 days.
Q37 5.	Write a query that shows which combination of CustomerTier and StoreType generates the highest average order value.
Q37 6.	Calculate the correlation proxy between UnitPrice and average rating per product (using window functions for rank correlation).
Q37 7.	Find all products that have had a price increase between their first and most recent order.
Q37 8.	Build a 'next best action' query: for each customer, identify the product category they have NOT yet purchased from that their demographic peers buy most.
Q37 9.	Write a query to compute the payback period for each store: months until cumulative revenue exceeded a hypothetical \$1M opening investment.
Q38 0.	Find employees with unusually high order cancellation rates compared to peers in the same store.
Q38 1.	Identify the 'long tail' in product sales: products that individually contribute < 0.1% to revenue but collectively account for what % of total products.
Q38 2.	Write a query that produces a complete customer health score from 1-100 combining: recency (30%), frequency (30%), monetary (25%), satisfaction/reviews (15%).

Chapter 5 — EXPERT Level Questions

Expert questions push you to combine every SQL technique you know, think about performance, and solve real-world data engineering problems. Allow up to 30 minutes each. Write your approach before coding.

5.1 Complex Analytics & Modelling

- Q38 3.** Build a complete RFM (Recency, Frequency, Monetary) segmentation model using CTEs and window functions, assigning each customer an RFM score and segment label.
- Q38 4.** Write a single query that calculates a cohort retention table: rows = join month, columns = month 0 through month 12, values = retention %.
- Q38 5.** Create a market basket analysis query that finds the top 20 product pairs purchased together, including support, confidence and lift metrics.
- Q38 6.** Write a query to detect sales anomalies: flag any day where revenue is more than 2 standard deviations from the 30-day rolling average.
- Q38 7.** Build a dynamic ABC inventory classification: classify products as A (top 70% revenue), B (next 20%), C (bottom 10%) using cumulative revenue share.
- Q38 8.** Write a query to compute the Net Promoter Score (NPS) equivalent using review ratings: % promoters (4-5) minus % detractors (1-2).
- Q38 9.** Construct a full profit and loss summary by store and quarter using pivot-style conditional aggregation.
- Q39 0.** Write a query that detects customers likely to churn, scoring them 1-100 based on recency, frequency drop and category diversity decline.
- Q39 1.** Build a store performance scorecard ranking each store on 5 KPIs: revenue, order count, avg basket size, return rate and customer satisfaction, then rank stores on a composite score.
- Q39 2.** Write a query to simulate a simple product recommendation engine: for a given customer, find the top 5 products bought by similar customers (who share at least 2 purchased products) but not yet bought by the target customer.
- Q39 3.** Use a recursive CTE plus window functions to flatten and annotate the full 5-level employee hierarchy, showing each person's level, their path from root, and their subtree headcount.
- Q39 4.** Write a gaps-and-islands query to find periods (consecutive date ranges) where a given store had zero orders.
- Q39 5.** Using window functions, identify the exact point in time when each customer's cumulative spending crossed the \$1000, \$5000 and \$10000 milestones.
- Q39 6.** Write a query to compute a rolling 90-day customer LTV and flag customers whose LTV trajectory is declining (the 30-day average is less than the 60-day average which is less than the 90-day average).
- Q39 7.** Create a seasonality index for each product: divide each month's average sales by the overall monthly average, returning values > 1 for above-average months and < 1 for below-average months.

5.2 Performance & Optimisation

Q39 8.	Given a query that scans the full Orders table to find a customer's last order, rewrite it to be index-friendly, explain what index you would create and why.
Q39 9.	The query <code>SELECT * FROM Orders JOIN Customers ...</code> runs slowly on 10 million rows. Identify at least 4 specific optimisation strategies and rewrite the query applying them.
Q40 0.	Explain and demonstrate the difference between a correlated subquery and a CTE-based equivalent for finding the top product per category. Compare their execution approach.
Q40 1.	Design and write the T-SQL to implement a slowly changing dimension (SCD Type 2) for the Customers table to track tier changes over time.
Q40 2.	Write a T-SQL stored procedure that accepts a date range and store ID, and returns a full sales summary report with error handling (TRY/CATCH), parameter validation and meaningful output messages.

5.3 Advanced Set Operations & PIVOT

Q40 3.	Use INTERSECT to find customers who ordered in both Q1 2022 and Q1 2023.
Q40 4.	Use EXCEPT to find products sold in 2022 but not in 2023.
Q40 5.	Combine UNION ALL, GROUP BY and ROLLUP to create a multi-level revenue summary by Region, Store and Month.
Q40 6.	Use GROUPING SETS to produce a single result set showing revenue by: (Category), (Store), (Category, Store), and a grand total row.
Q40 7.	Use PIVOT to create a cross-tab showing total revenue by Store (rows) and Quarter (columns) for the year 2023.

5.4 Advanced DML & Transactions

Q40 8.	Write a transaction that transfers 50 units of ProductID=5 from StoreID=1 to StoreID=2 in the Inventory table, with a ROLLBACK if either store's quantity would go negative.
Q40 9.	Write a MERGE statement that upserts inventory: if a (ProductID, StoreID) pair exists update the quantity, otherwise insert a new record.
Q41 0.	Write a query using OUTPUT clause to log all deleted orders (Status='Cancelled' and older than 1 year) into an OrderArchive table while deleting them from Orders.
Q41 1.	Implement a row-versioning audit trail: write a trigger on the Orders table that inserts a record into an OrderAudit table on every UPDATE, capturing old and new values.
Q41 2.	Write a query using CROSS APPLY with a table-valued function to parse and explode a comma-separated list of product tags stored in a column, returning one row per tag per product.

5.5 Real-World Scenarios

Q41 3.	A customer service rep needs to look up a customer by partial name or email. Write a single query that accepts a search term and returns matching customers with their last order date and total spend — optimised for partial match performance.
---------------	---

Q41 4.	Write a query that generates a daily operational dashboard for store managers, showing: today's order count, revenue, average basket, top 3 products sold, and comparison to the same day last week.
Q41 5.	Write a query that identifies supplier risk: flag suppliers where > 20% of their products have a return rate > 15% or average rating < 2.5.
Q41 6.	Create a query to automatically reorder products: list all products where StockQuantity <= ReorderLevel, calculate the suggested order quantity (3 * average monthly sales), and join to supplier contact info.
Q41 7.	Write a complete multi-CTE query that calculates employee bonuses: Sales employees get 2% of their attributed revenue, other employees get a flat 5% of salary, with a cap of \$10,000. Return employee name, department, base salary, bonus amount and new total compensation.
Q41 8.	Using window functions, find the single order that pushed each customer past their \$500 loyalty milestone.
Q41 9.	Write a query to compute the median order value per store without using PERCENTILE_CONT (use ROW_NUMBER and COUNT instead).
Q42 0.	Find customers where the gap between their 1st and 2nd order was longer than 90 days but subsequent gaps were shorter — indicating slow initial adoption then high engagement.
Q42 1.	Using LAG and LEAD together, classify each month's revenue as 'Peak' (higher than both neighbors), 'Trough' (lower than both), or 'Slope' (between two peaks or troughs).
Q42 2.	Write a query that finds the optimal product bundle of 3 products (triplet) that appears most frequently together in orders, returning the triplet and its co-occurrence count.
Q42 3.	Explain with a concrete example from RetailDB why SELECT DISTINCT is sometimes slower than GROUP BY for de-duplication.
Q42 4.	Write two equivalent versions of 'find customers with no orders' — one using NOT IN, one using NOT EXISTS — and explain which performs better and why.
Q42 5.	Demonstrate the difference between WHERE and HAVING using the Orders and OrderDetails tables with a concrete business question.
Q42 6.	Using the RetailDB schema, explain a scenario where an INNER JOIN would silently drop important data and how to fix it with a LEFT JOIN plus NULL check.
Q42 7.	Write an example that shows why using a function on a column in a WHERE clause (e.g., YEAR(OrderDate) = 2023) can prevent index usage and provide the sargable alternative.

Appendix — SQL Server Quick Reference

Window Function Syntax

<code>RANK() OVER (PARTITION BY col ORDER BY col)</code>
<code>DENSE_RANK() OVER (PARTITION BY col ORDER BY col)</code>
<code>ROW_NUMBER() OVER (PARTITION BY col ORDER BY col)</code>
<code>NTILE(n) OVER (ORDER BY col)</code>
<code>LAG(col, n) OVER (ORDER BY col)</code>
<code>LEAD(col, n) OVER (ORDER BY col)</code>
<code>SUM(col) OVER (PARTITION BY col ORDER BY col ROWS BETWEEN ...)</code>
<code>FIRST_VALUE(col)OVER (PARTITION BY col ORDER BY col)</code>
<code>PERCENT_RANK() OVER (ORDER BY col)</code>
<code>CUME_DIST() OVER (ORDER BY col)</code>

CTE Syntax

<code>WITH cte_name AS (SELECT ... FROM ...)</code>
<code>SELECT * FROM cte_name;</code>
<code>-- Multiple CTEs:</code>
<code>WITH cte1 AS (...), cte2 AS (...) SELECT ...</code>
<code>-- Recursive CTE:</code>
<code>WITH RECURSIVE cte AS (anchor UNION ALL recursive_part) SELECT ...</code>

Useful T-SQL Date Functions

<code>GETDATE() -- current datetime</code>
<code>DATEADD(unit, n, date) -- add interval</code>
<code>DATEDIFF(unit, d1, d2) -- difference</code>
<code>DATENAME(part, date) -- name of part</code>
<code>DATEPART(part, date) -- numeric part</code>
<code>FORMAT(date, pattern) -- formatted string</code>
<code>EOMONTH(date) -- last day of month</code>
<code>DATEFROMPARTS(y,m,d) -- build a date</code>

Common String Functions

<code>LEN(str) -- length</code>
<code>LEFT / RIGHT(str, n) -- substring from ends</code>
<code>SUBSTRING(str, s, len) -- mid substring</code>
<code>CHARINDEX(sub, str) -- find position</code>
<code>REPLACE(str, old, new) -- replace</code>
<code>UPPER / LOWER(str) -- case</code>
<code>LTRIM / RTRIM / TRIM -- whitespace removal</code>

CONCAT(a, b, ...) -- join strings
STRING_AGG(col, sep) -- aggregate to string
PATINDEX(pat, str) -- pattern position

Your SQL Mastery Roadmap

Phase	Questions	Focus	Goal
Foundation	Q1–Q95 (Easy)	SELECT, Filter, Aggregate, Sort	Zero syntax errors, fast queries
Development	Q96–Q280 (Medium)	JOINS, Subqueries, Functions	Multi-table fluency
Mastery	Q281–Q530 (Hard)	Windows, CTEs, Analytics	Write production-quality queries
Expert	Q531+ (Expert)	Performance, Architecture, Modelling	Think like a data engineer

Total Questions: 580+

Easy: 95 | Medium: 185+ | Hard: 250+ | Expert: 50+

Practice daily. Read execution plans. Rewrite every query 3 ways. You will master SQL.